

Práctica 3. Entradas y salidas digitales (GPIO)

(Duración: 1 sesión)

Introducción

En esta práctica aprenderá a programar la Raspberry Pi en Python para que lea y genere señales digitales. Para ello usará la tarjeta de expansión iMAT HAT e interactuará con los pulsadores y diodos LED integrados, así como con algún circuito adicional montado en la placa de prototipado.

Objetivos

Al finalizar la práctica, el alumno será capaz de:

- Configurar y usar los pines GPIO de la Raspberry Pi como entradas o salidas digitales.
- Entender la diferencia entre la gestión de entradas por nivel y por flanco.
- Manejar las librerías `RPi.GPIO` y `gpiozero`.
- Montar pulsadores externos en la placa de prototipado y conectarlos a la Raspberry Pi.
- Resolver problemas sencillos en los que intervengan una unidad de procesamiento y puertos digitales.

Evaluación

Aparte del trabajo previo, que el profesor revisará al principio de la sesión de laboratorio, una semana después de la finalización de la práctica cada grupo entregará un **informe** que deberá contener las respuestas a las cuestiones que se plantean en el enunciado así como el código desarrollado siguiendo las directrices que se indican en la normativa de laboratorio.

Para obtener la puntuación máxima **no es necesario** completar los apartados opcionales. Se incluyen en el enunciado por si quiere profundizar más en la materia aunque, evidentemente, se valorará su esfuerzo si los hace y los incluye en el informe.

Trabajo previo

Lea con atención el enunciado de la práctica y tome nota de las dudas que le surjan para preguntar al profesor. Intente también preparar los apartados señalados con el símbolo , incluyendo cómo hay que realizar los montajes en la placa de prototipado.

1. Conociendo los puertos

En esta práctica vamos a utilizar los ocho diodos LED de la iMAT HAT, que están conectados en configuración *pull-down* —activos a nivel alto— a los pines GPIO20 a GPIO27 de la Raspberry Pi, y los cuatro pulsadores (GPIO7, GPIO16, GPIO17, GPIO19). Los pulsadores están montados en *pull-up* —activos a nivel bajo—, aunque en la primera práctica no soldó la resistencia porque la Raspberry Pi la conecta internamente si se configura a través del código. Consulte en la sección de laboratorio de Moodle el esquema de la tarjeta de expansión si necesita más detalles acerca del conexionado.

Para ir calentando, conéctese por SSH a la Raspberry Pi y cree un archivo de Python en el directorio `Documents ▶ Lab03`. Copie y ejecute el Código 1 y después responda a las siguientes preguntas:

- ¿Qué hace el programa?
- Configure GPIO20 como entrada en lugar de como salida. ¿Qué sucede? ¿Es coherente?

```
import RPi.GPIO as GPIO

def main() -> None:
    GPIO.setmode(GPIO.BCM) # Nombre de los pines: Número GPIO

    # Configurar entradas (GPIO.IN) y salidas (GPIO.OUT)
    GPIO.setup(20, GPIO.OUT) # GPIO20 como salida

    # Definir valores iniciales de los pines
    # Nivel alto: GPIO.HIGH, True o 1
    # Nivel bajo: GPIO.LOW, False o 0
    GPIO.output(20, GPIO.HIGH)

    while True:
        pass

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        GPIO.cleanup() # Configurar los GPIO usados como entradas sin pull-up ni pull-down
    print("\r ")
```

Código 1. Ejemplo de manejo de salidas digitales con la librería `RPi.GPIO`. Como el bucle infinito del final del `main` que evita que el programa termine está vacío —no hay que actualizar ninguna salida después de la inicialización— se puede reemplazar por una llamada a `pause()`, que se importa del módulo `signal`.

2. Ejercitando la capacidad de creación

De ahora en adelante encontrará en los enunciados de las prácticas un apartado denominado “Ejercitando la capacidad de creación”, donde se plantean ejercicios sencillos para afianzar conceptos.

2.1. RPi.GPIO

En este caso, se trata de manejar entradas y salidas digitales usando la librería `RPi.GPIO` y distinguir cuándo hay que gestionar un pulsador —o en general una entrada— por nivel o por flanco.

- **Nivel:** Se utiliza cuando el estado de las salidas depende exclusivamente del valor actual de la entrada (la variable `pulsador` en la Figura 1). Se trata de una acción continua, las salidas conservan el valor adquirido **mientras** se mantenga la entrada.
- **Flanco:** Las salidas se modifican **cada vez que** se produce un cambio en la entrada. La forma de detectar este cambio (flanco) es crear una segunda variable (`pulsador_ant` en la Figura 1) que almacene el valor de la entrada en la ejecución anterior del bucle de `scan`. En un pulsador montado en *pull-up* se producen dos flancos, uno de bajada al pulsar y otro de subida al soltar. Lógicamente, en un pulsador en *pull-down*, sucede justo lo contrario, primero se observa el flanco de subida y luego el de bajada. En general, solo interesa modificar la salida una vez en todo el proceso de pulsar y soltar. Se suele preferir el primer flanco que se produzca para que el sistema transmita la sensación de que responde rápido. En el ejemplo de la Figura 1, cuando el valor de `pulsador` sea menor que el de `pulsador_ant`.

```

def main() -> None:
    ... # Definición de boton según la librería RPi o gpiozero

    pulsador = # Escriba aquí la función que lee dicha entrada
    while True:
        # Nivel
        pulsador = # Escriba aquí la función que lee dicha entrada
        if pulsador == 0:
            ...# Se ejecuta mientras pulsador sea 0 (varias veces)
        else:
            ...# Se ejecuta mientras pulsador sea 1 (varias veces)

        # Flanco bajada
        pulsador_ant = pulsador
        pulsador = # Escriba aquí la función que lee dicha entrada

        if pulsador_ant!=pulsador and pulsador == 0:
            ... # Se ejecuta solo cuando se detecta el flanco (una vez)

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        ...
  
```

Código 2. PseudoCódigo para detección por nivel o por flanco.

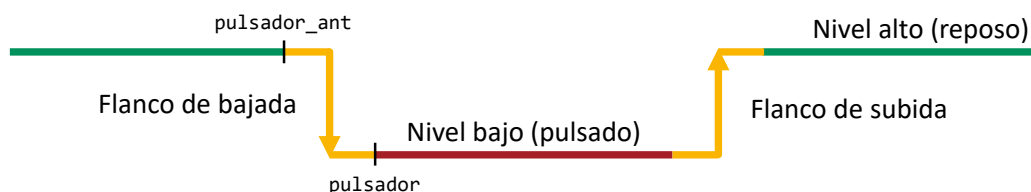


Figura 1. Cronograma de un pulsador en configuración *pull-up*. Cuando está en reposo, es decir, no está pulsado, la señal de entrada que recibe la Raspberry Pi es un '1' lógico (verde). Al pulsar la señal cambia y pasa a valer '0' (rojo). Cuando se deja de hacer fuerza, la señal retorna a nivel alto.

Después de esta breve introducción teórica, póngase manos a la obra y resuelva los siguientes tres ejercicios utilizando la librería **RPi.GPIO**¹:

1. Usando la función `GPIO.output()` solo una vez, escriba un programa que encienda los diodos LED de la iMAT HAT conectados a pines GPIO pares.
2. Modifique el código anterior para que se enciendan los LED pares solo **mientras** se presione el pulsador conectado a GPIO19. No olvide especificar el argumento `pull_up_down=GPIO.PUD_UP` en la función `GPIO.setup()` para conectar la resistencia de *pull-up* interna. Si quisiera activar una resistencia de *pull-down*, el valor del parámetro sería `GPIO.PUD_DOWN`.
3. Monte un pulsador externo en configuración *pull-up* en la placa de prototipado y conéctelo a GPIO14. Efectúe los cambios necesarios en el programa del punto 2 de modo que se mantenga la funcionalidad anterior y, además, **cada** activación de este nuevo pulsador alterne el estado de los LED impares. En otras palabras, si los LED impares están apagados y se presiona el pulsador, se encenderán; si se pulsa de nuevo, volverán a apagarse.

¹<https://sourceforge.net/p/raspberry-gpio-python/wiki/Examples/>

▲ **Observación:** Asegúrese de comprobar con un polímetro o con el osciloscopio que el pulsador está montado correctamente antes de conectarlo a la Raspberry Pi. Si provoca un cortocircuito la tarjeta se reiniciará y deberá esperar a que el sistema operativo arranque de nuevo.

2.2. gpiozero

Una vez que ya sabe utilizar la librería de bajo nivel RPi.GPIO, vamos a resolver los mismos ejercicios con **gpiozero**², que implementa clases sobre la anterior para manejar los diodos LED y los pulsadores de forma más sencilla.

Existe la posibilidad de utilizar eventos para gestionar los flancos. **En esta sección, cuando crea que necesita utilizar un flanco, deberá resolver el ejercicio de las dos formas obligatoriamente: comparando el valor actual de la entrada con el anterior y usando callbacks.** En la librería gpiozero, además de poder consultar el valor lógico del pulsador directamente con la propiedad `value`, están disponibles la propiedad `is_pressed`, que devuelve `True` cuando el pulsador está pulsado, y dos eventos `when_pressed` y `when_released`, que se disparan al pulsar y soltar el botón, respectivamente (Código 3). La librería determina automáticamente si pulsado es cuando la entrada está a nivel bajo o alto a partir del valor que tome el argumento `pull_up`, que se fija al construir el objeto (por defecto, `True`).

Los *callbacks* que se registran para los eventos —que como norma general solo deberían asociarse una vez antes del bucle de *scan*—, no reciben argumentos por defecto. Si se quiere pasar algún parámetro a estas funciones hay que utilizar una **función lambda**. Para evitar errores por manejarlas de manera incorrecta, se recomienda **capturar explícitamente todos los argumentos** como se hace en el ejemplo, es decir, asignando con un igual cada argumento que aparezca a la derecha de la palabra clave *lambda* —si hubiera más de uno se separan por comas— y antes de los dos puntos.

1. Usando la clase LED —no LEDBoard—, escriba un programa que encienda los diodos LED de la iMAT HAT conectados a pines GPIO pares.
2. Modifique el código anterior para que se enciendan los LED pares solo **mientras** se presione el pulsador conectado a GPIO19.
 - a) Utilice exclusivamente las clases LED y Button.
 - b) Emplee LEDBoard en lugar de LED.
3. Utilizando el pulsador externo en configuración *pull-up* que todavía debería tener conectado a GPIO14, en el conector interior, efectúe los cambios necesarios en el programa del punto 2a) de modo que se mantenga la funcionalidad anterior y, además, **cada** activación de este nuevo pulsador alterne el estado de los LED impares. En otras palabras, si los LED impares están apagados y se presiona el pulsador, se encenderán; si se pulsa de nuevo, volverán a apagarse.
 - a) Usando las variables anterior y actual para el pulsador GPIO14.
 - b) Usando callbacks y lamdas para capturar parámetros para el GPIO14.

²<https://gpiozero.readthedocs.io/>

```
from gpiozero import Button, LED

def al_pulsar() -> None:
    pass

def al_soltar(led: LED) -> None:
    pass

def main() -> None:
    button = Button(16, pull_up=True)
    led = LED(20)

    #Ejemplo asignación sin parámetros
    button.when_pressed = al_pulsar
    #Ejemplo asignación de parámetros, sin el lambda no funcionará bien
    button.when_released = lambda led=led: al_soltar(led=led)

    while True:
        pass

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\n ")
```

Código 3. Ejemplo de detección de flancos en los pulsadores con la librería gpiozero. Los *callbacks* no reciben argumentos por defecto, pero se pueden enviar utilizando una función lambda.

3. Aprendiendo a resolver problemas

En los enunciados también se cruzará habitualmente con una sección denominada “Aprendiendo a resolver problemas”, en la cual utilizará todo lo aprendido en los apartados anteriores para solucionar problemas algo más complejos, los típicos de algo fácil que se puede pedir en el examen.

En esta ocasión se le pide desarrollar un programa usando **gpiozero** que muestre un contador de 8 bits en binario en los diodos LED de la tarjeta de expansión. Considere que GPIO20 es el bit menos significativo y GPIO27, el más significativo. Cada vez que se presione GPIO7 se sumará una unidad, cada activación de GPIO16 restará una unidad y GPIO17 reiniciará la cuenta desde 0. Así, por ejemplo, si se pulsa cuatro veces GPIO7 y una GPIO16, se encenderán los LED conectados a GPIO20 y GPIO21, que representan 3 en binario.

⚠ Observación: Para poder utilizar el pulsador de GPIO7 hay que deshabilitar la interfaz SPI, que permite comunicarse con el chip MCP3008, el conversor A/D que soldó en la primera práctica y que usaremos más adelante en el curso. Escriba **sudo raspi-config** en el Terminal, navegue hasta **3 Interface Options** ▶ **I3 SPI** y seleccione **No**.

Para mostrar el contador necesitará separarlo en cada uno de los bits que componen su codificación en binario.

Aunque trabajemos habitualmente en decimal, todos los números se almacenan internamente en binario, por lo que se pueden utilizar operadores a nivel de bit directamente. El Código 4 almacena en la lista `bits` los 8 bits más bajos de la variable `contador`. El primer elemento de la lista se corresponde con el bit menos significativo. **Utilice dicho código para resolver el enunciado.**

- ¿Podría explicar en detalle qué hace esta línea de código e incluir la respuesta en el informe?

```
bits = [(contador >> bit) & 0x1 for bit in range(8)]
```

Código 4. Separar una variable entera en sus bits individuales.

4. Para los más intrépidos (Opcional)

Dada la importancia de conocer la dirección IP del adaptador Wi-Fi inalámbrico (`wlan0`) para poder conectarse a la Raspberry Pi en caso de no disponer de un servidor DNS, si termina todos los ejercicios anteriores antes del final de la sesión quizá quiera escribir un programa que permita mostrar los cuatro octetos por separado en los LED de la iMAT HAT. Los diodos LED empezarán apagados y al pulsar cada uno de los cuatro pulsadores se mostrará un octeto diferente. Considere que `GPIO20` se corresponde con el bit más significativo y `GPIO27` con el menos significativo (Tabla 1). Puede resolver el problema utilizando cualquiera de las librerías que ha aprendido durante la práctica. Lógicamente, también puede hacerlo con ambas...

Tabla 1. Octeto que hay que mostrar en función del pulsador que se presione y ejemplo con la IP 192.168.0.2.

Pulsador	Octeto	Decimal	GPIO							
			20	21	22	23	24	25	26	27
GPIO7	1º	192	1	1	0	0	0	0	0	0
GPIO16	2º	168	1	0	1	0	1	0	0	0
GPIO17	3º	0	0	0	0	0	0	0	0	0
GPIO19	4º	2	0	0	0	0	0	0	1	0

```
import socket

def leer_ip() -> str:
    st = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    try:
        st.connect(("10.255.255.255", 1))
        ip = st.getsockname()[0]
    except Exception:
        ip = "127.0.0.1"
    finally:
        st.close()

    return ip
```

Código 5. Función que devuelve la dirección IP.

5. Solo los curiosos llegan aquí (Opcional)

Casi todas las librerías que gestionan los pines de la Raspberry Pi disponen de un mecanismo ya implementado para lidiar con los flancos. En el caso de `RPi.GPIO`, hay que registrar un evento y asociarlo a una función (*callback*), que se llamará automáticamente cuando se produzca el flanco (Código 6). Este es un comportamiento avanzado que, si no se usa correctamente, puede dar lugar a errores difíciles de depurar. Si quiere, puede probarlo al finalizar la práctica e incluirlo como un ejercicio **opcional** en el informe, pero tenga en cuenta que comparar el valor actual con el anterior es más fácil de entender.

```
def flanco_subida(gpio: int) -> None:
    pass

def flanco_bajada(gpio: int) -> None:
    pass

GPIO.add_event_detect(gpio, GPIO.RISING, callback=flanco_subida)
GPIO.add_event_detect(gpio, GPIO.FALLING, callback=flanco_bajada)
```

Código 6. Método para asociar una función a la detección de un flanco de subida o bajada con la librería `RPi.GPIO`. Los *callbacks* reciben un único argumento con el número del pin GPIO que ha desencadenado la llamada.